CAPSTONE — DFJSP · UNITY + ML-AGENTS

HOW THE SIMULATION RUNS

One factory-floor episode, from CLI launch to the seven CSVs it leaves behind. Jobs route through machines and AGVs under either a fixed `DispatchingRule` or a trained policy, one decision at a time, until the shop clears or the run is declared stuck.

9 discrete actions — 8 PDRs + Random

2 decision types — Routing, Dispatch

6 job states — NeedsRouting → Exited

100,000s episode timeout

3,000s deadlock stall watchdog

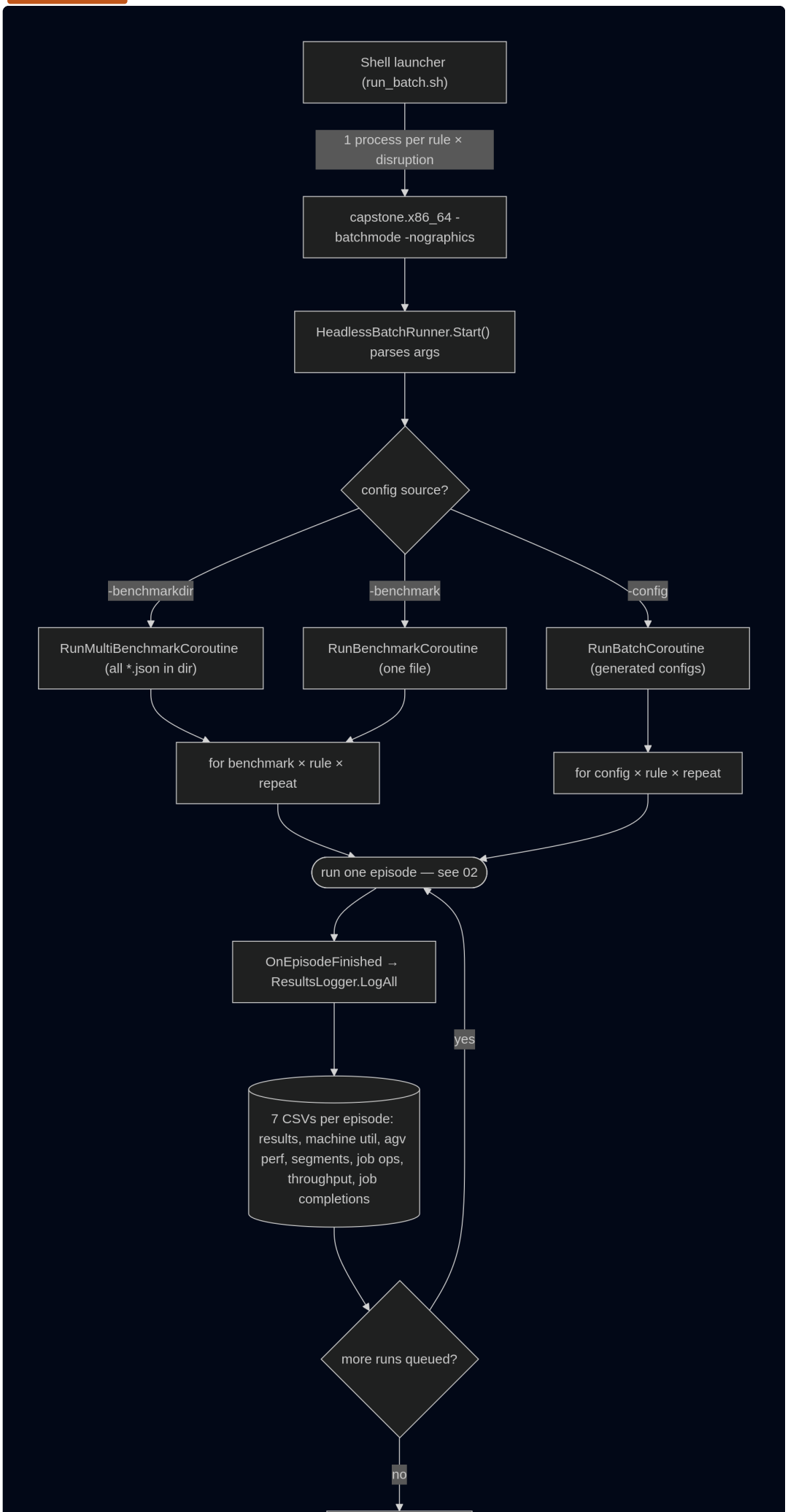
7 CSVs written per episode

---

01 BATCH PIPELINE

A shell script (`run_batch.sh`, `run_generated.sh`, or `run_agv_sweep.sh`) launches one headless Unity process per rule × disruption level. Inside each process, `HeadlessBatchRunner` reads its CLI args once, then drives every config/rule/repeat combination through the same episode loop before the process exits and the shell script merges the per-worker CSVs.

FLOWCHART

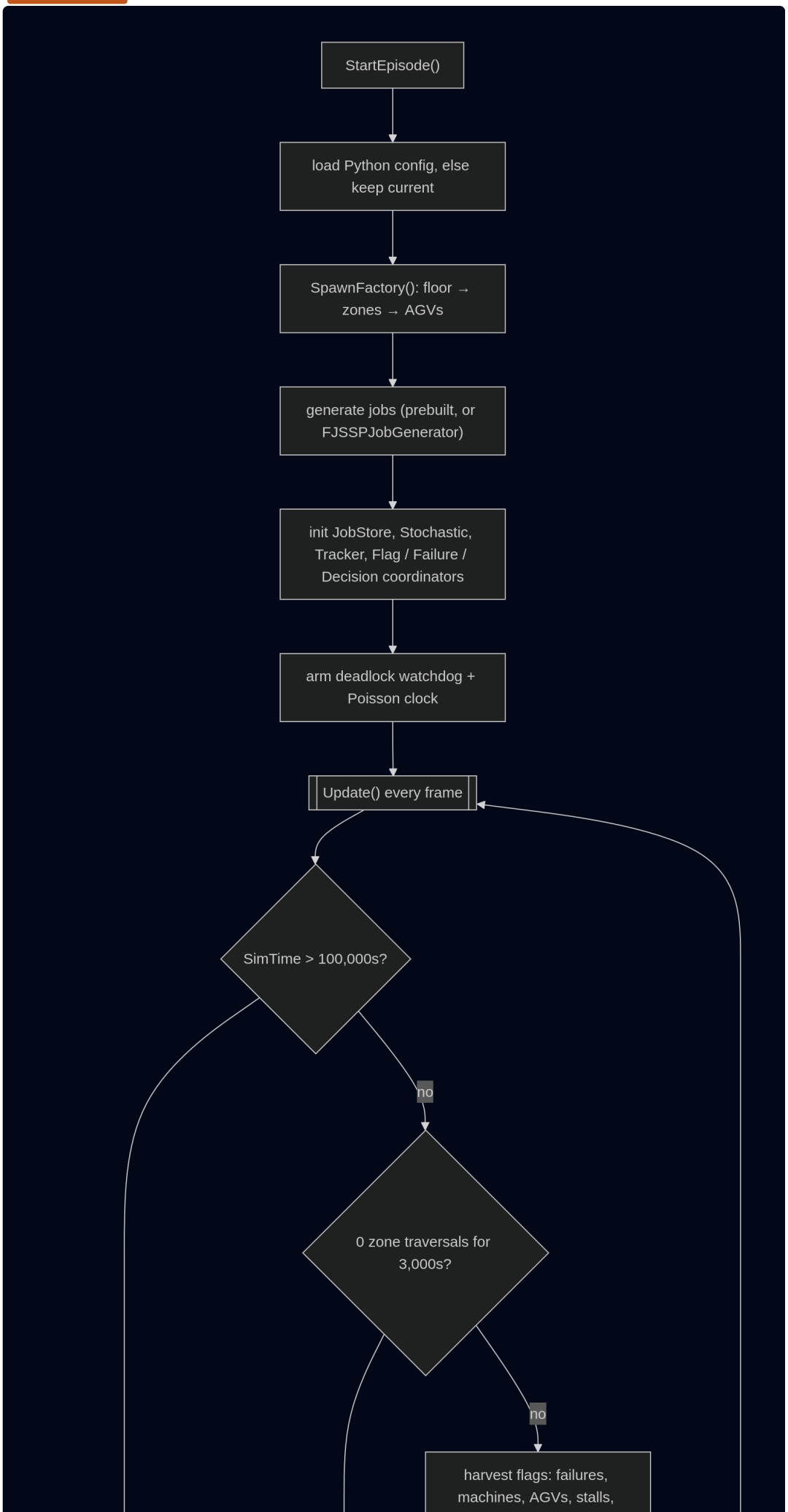

Shell launcher (run_batch.sh)
1 process per rule × disruption
capstone.x86_64 -batchmode -nographics
HeadlessBatchRunner.Start() parses args
config source?
-benchmarkdir
-benchmark
-config
RunMultiBenchmarkCoroutine (all *.json in dir)
RunBenchmarkCoroutine (one file)
RunBatchCoroutine (generated configs)
for benchmark × rule × repeat
for config × rule × repeat
run one episode — see 02
OnEpisodeFinished → ResultsLogger.LogAll
yes
7 CSVs per episode: results, machine util, agv perf, segments, job ops, throughput, job completions
more runs queued?
no

> **-baselinedrain** flips `FactoryOrchestrator.BaselineDrainMode` on before the loop starts. It's a batch-only shortcut for fixed-rule runs — see the branch in 03 — and must stay off for anything with a neural policy attached.

02 EPISODE LIFECYCLE

`FactoryOrchestrator.StartEpisode()` builds the floor (if not already spawned), generates or loads the job batch, and arms two independent clocks — a Poisson arrival clock for dynamic jobs and a deadlock watchdog. From there, `Update()` runs once per frame until one of three exits fires.

FLOWCHART


StartEpisode()
load Python config, else keep current
SpawnFactory(): floor → zones → AGVs
generate jobs (prebuilt, or FJSSPJobGenerator)
init JobStore, Stochastic, Tracker, Flag / Failure / Decision coordinators
arm deadlock watchdog + Poisson clock
Update() every frame
SimTime > 100,000s?
no
0 zone traversals for 3,000s?
no
harvest flags: failures, machines, AGVs, stalls,

process step

branch point

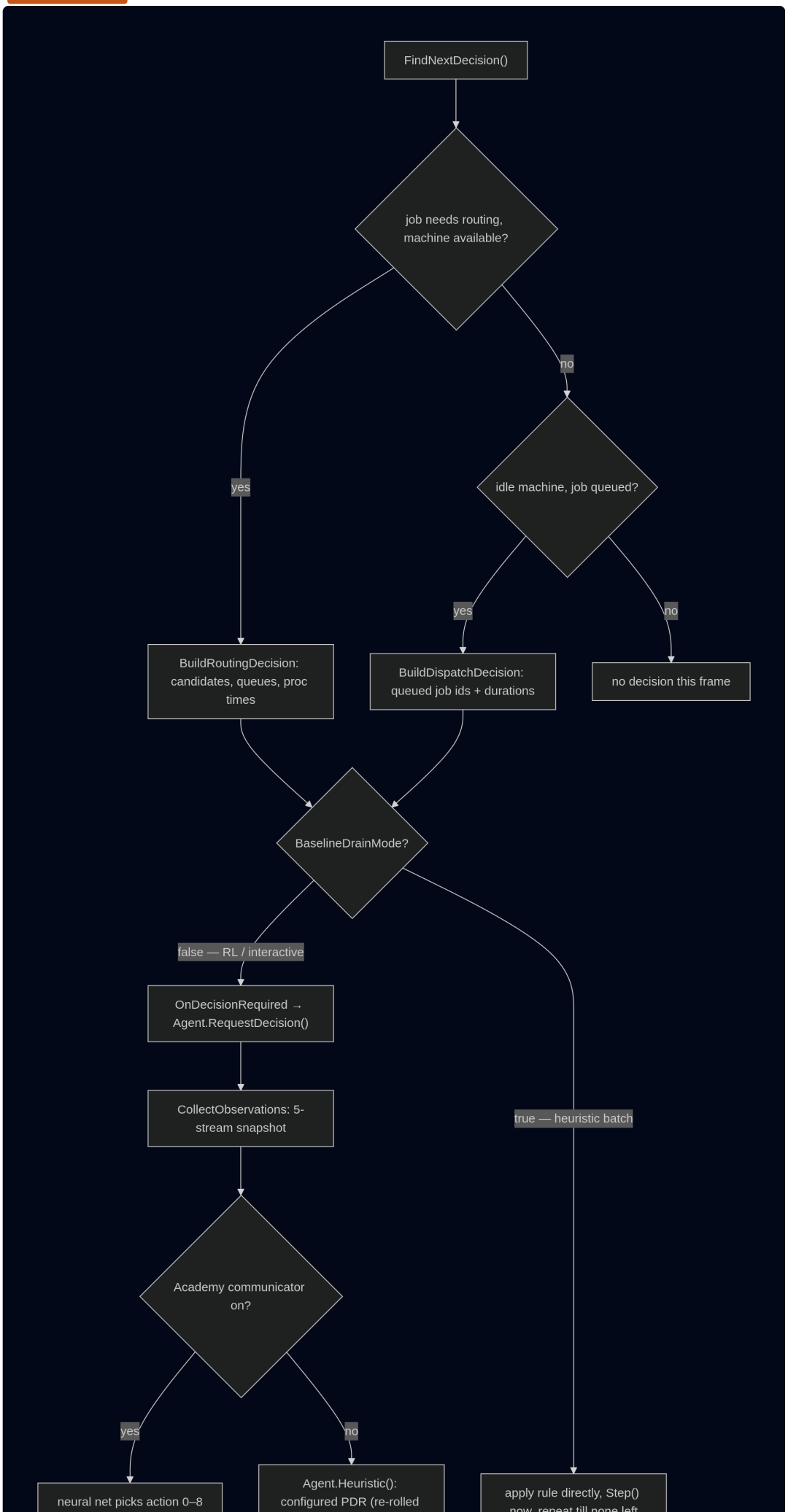
sub-flow / loop entry

Deadlock watchdog sums `TrafficZone.TraversalCount` across the whole network every frame. A true circular-wait in AGV zone reservations never self-resolves, so 3,000 stalled sim-seconds with zero traversals anywhere is treated as conclusive — the episode is cut short instead of burning to the 100,000s timeout.

03 DECISION LOOP

Every frame, `DecisionCoordinator.FindNextDecision()` looks for exactly one thing to resolve: a job that needs a machine (**Routing**), or an idle machine with something in its queue (**Dispatch**). What happens with that request next depends on which of three modes is active.

FLOWCHART


FindNextDecision()
job needs routing, machine available?
yes
no
idle machine, job queued?
yes
no
BuildRoutingDecision: candidates, queues, proc times
BuildDispatchDecision: queued job ids + durations
no decision this frame
BaselineDrainMode?
false — RL / interactive
true — heuristic batch
OnDecisionRequired → Agent.RequestDecision()
CollectObservations: 5-stream snapshot
Academy communicator on?
yes
no
neural net picks action 0–8
Agent.Heuristic(): configured PDR (re-rolled
apply rule directly, Step()

process step

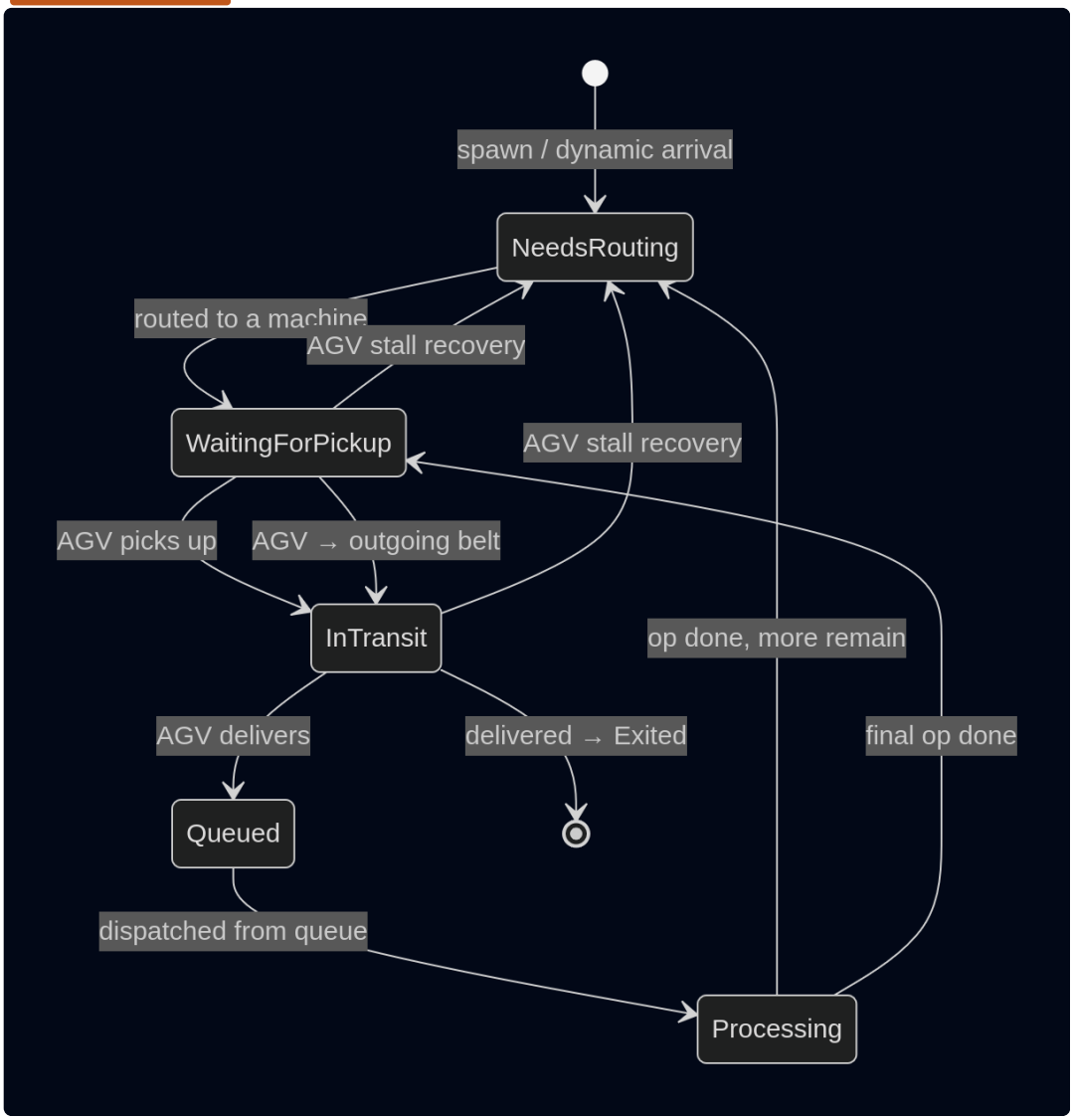
branch point

The 8 PDRs are `SPT_SMPT`, `SPT_SRWT`, `LPT_MMUR`, `LPT_SMPT`, `SRT_SRWT`, `SRT_SMPT`, `LRT_MMUR`, and `SDT_SRWT` — the same table drives both `SelectMachine` and `SelectJob`. **Random** isn't sampled once per episode: `DispatchingEngine` re-rolls one of the other 8 at every single decision when the configured rule is `Random`.

04 JOB STATE MACHINE

Every job cycles through this loop once per operation. A finished op either sends the job back to `NeedsRouting` for its next operation, or — on the last operation — out through the AGV network to `Exited`. AGVs can be pre-dispatched up to `PreDispatchLeadTime` seconds before a machine finishes, so pickup is often already waiting when a job clears `Processing`.

STATE DIAGRAM



> The two **stall-recovery** edges only fire when an AGV suspects a traffic-zone deadlock and returns to parking on its own (`AGVController.HandleZoneStall`). The job loses its AGV assignment and re-enters routing; a job that was only pre-dispatched (still `Processing`, never picked up) just loses that claim and stays put.

05 SOURCE MAP

Where each piece of the diagrams above actually lives.

REFERENCE

COMPONENT	ROLE	FILE
FactoryOrchestrator	Episode lifecycle, per-frame Update loop, Step()	`Simulation/FactoryOrchestrator.cs`
HeadlessBatchRunner	CLI parsing, batch/benchmark coroutines	`Simulation/HeadlessBatchRunner.cs`
DecisionCoordinator	Finds and builds the next Routing/Dispatch request	`Simulation/DecisionCoordinator.cs`
DispatchingEngine	PDR action table, SelectMachine / SelectJob	`Simulation/DispatchingEngine.cs`
SchedulingAgent	ML-Agents bridge — observations, heuristic, action	`Simulation/SchedulingAgent.cs`
ObservationBuilder	Builds the 5-stream observation snapshot	`Simulation/ObservationBuilder.cs`
FlagHarvester	Drains per-frame machine/AGV state flags	`Simulation/FlagHarvester.cs`
FailureCoordinator	Weibull machine failures + repair scheduling	`Simulation/FailureCoordinator.cs`
StochasticEventManager	Poisson arrivals, failure sampling	`Simulation/Stochastic/StochasticEventManager.cs`
TrafficZoneManager	Zone graph, AGV reservations, congestion	`Simulation/FactoryLayout/TrafficZoneManager.cs`
AGVPool / AGVController	Fleet management, pickup/delivery, pre-dispatch	`Simulation/AGV/*.cs`
JobStore / JobData	Job state machine, per-op timing fields	`Simulation/Jobs/*.cs`
EpisodeTracker	Aggregates stats into the EpisodeRecord	`Simulation/EpisodeTracker.cs`
ResultsLogger	Writes the 7 per-episode CSVs	`Simulation/Logging/ResultsLogger.cs`

COMPONENT	ROLE	FILE
run_batch.sh	Parallel launcher + CSV merge, per disruption level	`linux_server/run_batch.sh`
